

5LSL0 Machine Learning for signal processing - assignment 2

Gheorghiu Cristian Stefan*

Student number: 2036967

1 EXERCISE 2.3

1.1 2.3.a Requirement

How do strided downsampling layers in a CNN affect the receptive field?

1.2 2.3.a Answer

The growth of receptive field across layers of a CNN based network becomes more and more accelerated as the stride increases. This is particularly demonstrated by the example below, where drawing upon the theoretical receptive field formula 1 we can observe how the aforementioned duality materializes.

$$TRF_L = 1 + \sum_{i=1}^L [(K_i - 1) * \prod_{j=1}^{i-1} s_j] \quad (1)$$

For 2 assume a CNN with 2 layers: First layer kernel size is 3x3, stride is 3 and padding is zero. Second layer kernel size is 3x3m stride is 1 and padding is zero

$$\begin{aligned} TRF_{l=1} &= 1 + [(3 - 1) \times 1] = 3 \\ TRF_{l=2} &= 1 + [(3 - 1) \times 2] = 4 \\ TRF &= TRF_{l=1} + TRF_{l=2} = 7 \end{aligned} \quad (2)$$

For 3 assume the same CNN architecture as above but WITHOUT STRIDING.

$$\begin{aligned} TRF_{l=1} &= 1 + [(3 - 1) \times 1] = 3 \\ TRF_{l=2} &= 1 + [(3 - 1) \times 1] = 3 \\ TRF &= TRF_{l=1} + TRF_{l=2} = 6 \end{aligned} \quad (3)$$

1.3 2.3.b Requirement

A CNN without strided downsampling layers is equivariant to the symmetry group of all integer-pixel translations. If we use two strided downsampling layers in our CNN, what symmetry group is the network equivariant to?

1.4 2.3.b Answer

As showed in [3] [4] a two strided downsampling CNN layer becomes equivariant to a particular spectrum of translations, primarily constituted from shifts operations as multiples of 2 pixels. More generally, architectures carrying a strider higher than one, can generalize only to lattice sizes that are multiple of the stride combinations.

1.5 2.3.c Requirement

The simulator quantizes the delays to integers, meaning nearby locations will result in the same simulation results. Create a figure showing which areas result in the same simulation results for (x, y) source locations (0, 50), (-100, 50), (0, 150) and (100, 150)cm. Explain your process

*e-mail: c.gheorghiu1@student.tue.nl

1.6 2.3.c Answer

The process of determining the areas (e.g., pixel/space coordinates) for which the simulated results (e.g., time delays calculated based on the distance between fixed positions and microphone locations) are considered similar to time delays derived based on distinct coordinates (e.g., space coordinates) is linked to the relative time delays. As Figure 1 depicts, such zones where discrepancies between the relative time delays do not exist were coloured with green. This space colourisation takes place when the space differences relative to the first microphone are equal to the fixed differences (e.g., (0, 50), (-100, 50), (0, 150) and (100, 150) cm) again to the first microphone.

The prevalent red colour reflects that sound transmission from the vast majority of space locations will result in arrival delays that are different from what we would usually get if the same signal were forwarded from any of the four known sources. To have this experiment hold true, saving the relative time delays (the difference in arrival time between Mic 1 and the other three mics) is crucial, as I discovered earlier; the absolute travel time changes as you move away.

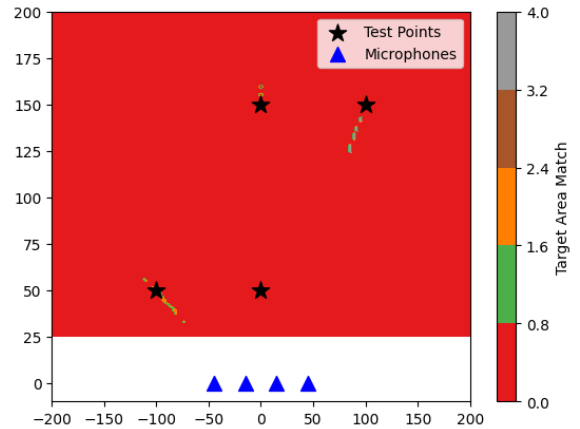


Figure 1: Spatial Quantization of Relative Time Delays in a Four-Microphone Array

1.7 2.3.d Requirement

Design a 1D neural network with two outputs: the estimated x- and y-coordinate of the source. Explain your design choices.

1. Implement and train your model. Plot both the loss graph and the average validation accuracy in centimeters over time.
2. (optional:) You are encouraged to iterate on your design and change things if you realize your design choices were sub-optimal. Explain the changes you made to your original design, the reasoning behind those changes, and whether this resulted in improvement.

1.8 2.3.d Answer

1. Check both, Figure 2 and Figure 3

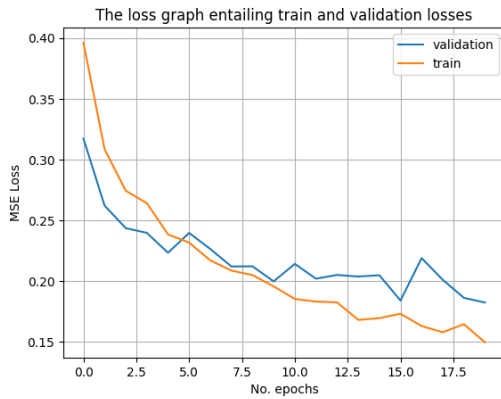


Figure 2: The loss graph entailing train and validation losses

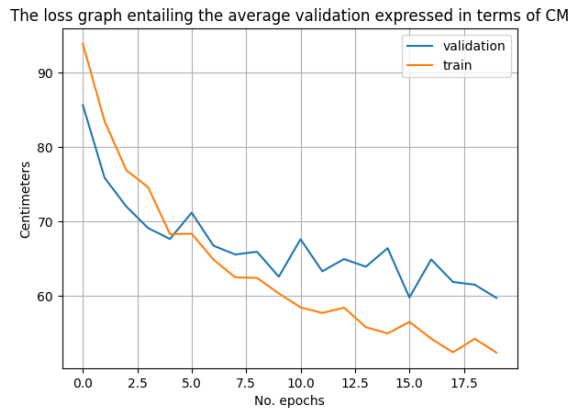


Figure 3: The loss graph entailing the average validation expressed in terms of CM

2. A hardly identifiable flaw in the initial architecture not only kept the model performance within an unsatisfactory region (e.g., under 30% accuracy) but also caused its learning curve to lack any concrete downward trend and to be extremely volatile. I initially suspected the network not to be powerful enough (e.g., to not have enough capacity) and decided to keep on using the same track/.mp3 file to promote overfitting to see whether the model could actually 'learn'. The trial turned out to yield insightful results based on which I counted out the model's architecture from the list of possible culprits. This made me reorient my attention to the custom Dataloader, where I noted by printing out data-related information (e.g., shapes) that the batch dimension remained fixed at 28 (which is the maximum number of training files) irrespective of how large the batch-size variable is. This flaw arose from how I design the magic function `def __len__(self):` to return the length based on the number of training files counted (a maximum of 28).

Such adjustments made for a stable (e.g., lack of spikes/volatility) but not satisfactory convergence, as the model's performance was far behind the 65% accuracy bar. This prompted me to explore some feature transformations I could implement and execute during training without accidentally altering any type of information. I came up with `AudioPool(nn.Module)`: that encompasses two mean-max pooling operations, meant to retrieve the loudest occurrence/activity over channels and avoid capturing peaks provoked by noisy artifacts.

1.9 2.3.e Requirement

Calculate the receptive field of your model.

1. Verify this result experimentally; explain your process and include a figure.
2. Does the theoretical result match the receptive field that you find in practice?

1.10 2.3.e Answer

1. To get up to speed with the class pace, I called in the torch-scan module **summary** and enabled its argument to true (e.g., **receptive field=True**) which adds on the summary table a fifth column encompassing the receptive fields corresponding to each layer in the network. Summing up all the elements, from bottom to top we obtain a **TRF (theoretical receptive field) of 19**. The prebuilt function has at its core the formula 1 I used previously. The results can be seen in the Table 1.
2. As demonstrated in [1][2][6], but also enforced by the result revealed by my hand-made calculations, the **TRF as calculated before does not actually match the one I found in practice**. This divergence has its roots deeply buried in the expected/imagined growth of the receptive field (which we assume to be of a linear nature, expanding as more layers L are being added), whereas experiments are showing that the ERF's effective radius (measured by the standard deviation of its Gaussian shape, σ) grows proportionally to the square root of the depth, $\sigma \propto \sqrt{L}$.

Table 1: Model Architecture Summary

Layer	Type	Output Shape	Param #	Receptive Field
defaultcnn	DefaultCNN	(-1,2)	0	19
└ n	Sequential	(-1,128)	0	19
0	Conv1d	(-1,32,4800)	256	19
1	BatchNorm1d	(-1,32,4800)	129	13
2	LeakyReLU	(-1,32,4800)	0	13
3	Conv1d	(-1,64,2400)	10,304	13
4	BatchNorm1d	(-1,64,2400)	257	5
5	LeakyReLU	(-1,64,2400)	0	5
6	Conv1d	(-1,64,1200)	20,544	5
7	BatchNorm1d	(-1,64,1200)	257	1
8	LeakyReLU	(-1,64,1200)	0	1
9	AudioPool	(-1,128)	0	1
└ fc1	Linear	(-1,32)	4,128	1
└ activ	LeakyReLU	(-1,32)	0	1
└ drop1	Dropout	(-1,32)	0	1
└ fc2	Linear	(-1,2)	66	1

Trainable params: 35,618

Non-trainable params: 0

Total params: 35,618

Model size (params + buffers): 0.14 Mb

Framework & CUDA overhead: 0.00 Mb

Total RAM usage: 0.14 Mb

Floating Point Operations on forward: 105.07 MFLOPs

Multiply-Accumulations on forward: 52.15 MMACs

Direct memory accesses on forward: 52.18 MDMAs

1.11 2.3.f Requirement

Once you're done developing your model with the training and validation set, run the model on your test set. Explain your process of evaluating the performance and report your results. Include a representative figure visualizing the performance. Also comment on how the test-set performance compares to the validation set performance, and how this compares to the quantization ambiguity found in (c).

1.12 2.3.f Answer

To evaluate different architectures and optimize hyperparameters, the model was initially trained on the training set and tuned using the validation set. To put the model's performance evolution in a clearer perspective and demonstrate convergence stability, I then loaded the pre-trained weights and ran the model for 15 epochs over the test set. As shown in Figure 4, performance steadily improves during the training-validation phase and remains within the expected optimal threshold (defined by an MSE loss of 0.22 or lower) when evaluated on the test set. **However, usually the test set is predestinated to one-pass experiment which in this case yielded the following figures: MSE Loss: 0.2237 Test Error in CM: 65.64 cm.**



Figure 4: Model performance evolution

On a slightly different note, the model performance as portrayed by the aforementioned figures is, without a shadow of a doubt, negatively influenced by the quantization ambiguity. If several particular adjustments (such as increasing the bit depth or artificially injecting the noise) were made then we would have attained even a nicer performance than the one shown above. However, for the first part of the assignment, I was more interested in describing the limitation that stems from this kind of quantization ambiguity, which I managed to translate mathematically as shown below in 4.

$$T_s = \frac{1}{f_s}$$

$$T_s = \frac{1}{48,000}$$

$$T_s \approx 0.000020833 \text{ seconds (or } 20.83 \mu\text{s)}$$

$$D = 343 \text{ m/s} \times 0.000020833 \text{ s} = 0.0071458 \text{ meters}$$

$$D = 0.0071458 \text{ m} \times 1000 = \mathbf{7.1458 \text{ mm}}$$

This 7.15 mm value represents the absolute theoretical threshold resolution limit of the adopted sampling rate of 48kHz. If the sound source moves a distance smaller than this limit, the change in the time delay is just too brief to cross into an adjacent digital sample bucket s_{i-1} . This sample index that remains unchanged is materialized in identical input tensors making movements less than 7.15 mm "invisible" to the model.

1.13 2.3.g Requirement

Design and train a 1D neural network that can locate between 1 and 3 sources simultaneously. For example, you can use a network where the coordinates of the closest source are estimated by the first two outputs, the second source by the next two, and so forth, with one final network output estimating how many sources are active. Again, explain your design choices and visualize the training process in a figure.

- (optional:) You are encouraged to iterate on your design and change things if you realize your design choices were sub-optimal. Explain the changes you made to your original design, the reasoning behind those changes, and whether this resulted in improvement

1.14 2.3.g Results

- The training process can be visualized in the following Figures 5 and 6

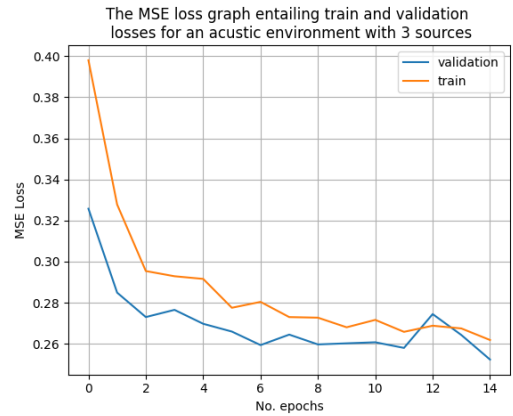


Figure 5: Train and Validation losses for an environment with 3 acoustic sources

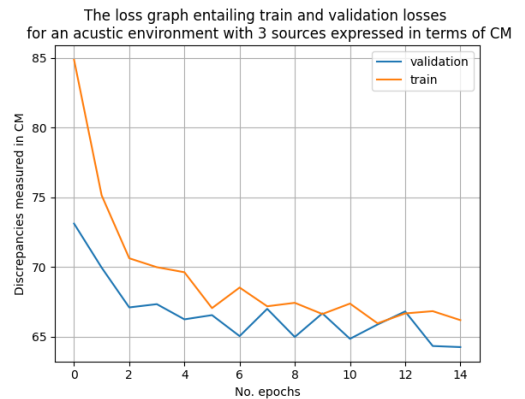


Figure 6: Train and Validation losses expressed in terms of CM for 3 acoustic sources

- Laying out through an enumeration the changes I made across multiple sectors, ranging from the network architecture itself to the custom data loader all the way to the training loop in order to accommodate the task at hand:
 - Automatic adjustment of the learning rate, commonly known as a learning rate scheduler, using the module named 'ReduceLROnPlateau' equipping 2 patient

epochs (patience=5). Reduce the learning rate when a metric has stopped improving.

- (b) By setting the argument to None (reduction=None in the nn.MSELoss()), the adoption of a dynamic mask becomes possible. Since the network will always generate a fixed number of predictions (e.g., 6 pairs of coordinates) irrespective of how many sources do actually exist, a penalty must be applied only for the existent sources.
- (c) Typical adjustments (such as $p = (\text{kernel size} - 1)/2$) of CNN hyperparameters (e.g., kernel size, padding and stride) will not make result in rich gains. Starting off with a kernel size that is large enough to capture the maximum delay between the furthest microphone and the source becomes a strictly required adjustment.
- (d) Adopting a sorting rule (e.g. sorting by relative distance between sources and microphones along a single dimension X) that is not restructuring the hierarchical order of the prediction indices turned out to be a game changer as, the model was able to break through the 30% accuracy plateau much faster than before. On top of that, to provide the network with concise insights regarding to how well it can predict either the pair of coordinates or the number of sources, I created a composite error placing as much weight (95%) as possible on the coordinates.

1.15 2.3.h Requirement

If we order our outputs by distance from the microphones as suggested above, what may happen if two sources in a training sample are at similar distances? Is that a problem, and if so, how could we fix it?

1.16 2.3.h Answer

If two sources are at similar distances, slight fluctuations in the model's predictions during training can cause their relative ordering to flip continuously. This is a significant problem because it causes label swapping. The target labels assigned to specific output nodes will suddenly change between training steps. Consequently, at loss computation time, the correspondence between the predictions and the ground truth is destroyed. The loss function will compare a prediction to the wrong source target, yielding massive, unstable penalties that confuse the network and derail the learning curve. To resolve this, we can abandon fixed heuristic ordering and instead use Permutation Invariant Training (PIT). In PIT, the network computes the loss for all possible permutations of assignments between the predicted outputs and the ground truth targets. It then backpropagates the error using only the permutation that yields the minimum loss, entirely bypassing the need to sort by distance.

1.17 2.3.i Requirement

Once you're done developing your model, run the model on your test set. Explain your process of evaluating the performance in different scenarios and report your results. Include representative figures visualizing the performance

1.18 2.3.i Answer

The vast majority of adjustments made during the training-validation phase yielded enough of a positive and strong change to make further augmentations or adoptions meaningless. Therefore, based on a known suite of hyperparameters, table 2 represents a solid outline of the model performance on unseen data for a varying number of sources. Telling exactly how many acoustic sources the model can handle simultaneously without difficulty (or without having the prediction error deviate from what we've seen during training and testing) remains a fairly difficult task given the minor discrepancies we see in the table. As worth trying, the follow-up experiment will

consist of increasing the number of sources until the model prediction error completely derails and labeling the breaking point (e.g., 10 sources) as the upper bound of the optimal domain. Furthermore, random assignment of the number of sources might not be an ideal training paradigm. A more favorable and worth applying approach would be to put in place a mechanism that ensures a even exposure of the network to any of the three settings (e.g., 1, 2 or 3 sources)

Number of Sources	Final Test MSE Loss	Final Test Error (cm)
1	0.2633	70.43
2	0.2605	70.41
3	0.2635	71.11

Table 2: Final model evaluation on the unseen test set, detailing Mean Squared Error (MSE) and Centimeter (CM) error across varying numbers of active acoustic sources.

1.19 2.3.j Requirement

Simulate the full recording of both bird sounds in the test set as sources located at (0, 50)cm and (100, 150)cm respectively. Plot the estimated location error over time for both. Does the location error correlate with the sound volume of the recordings at different times? Comment on your findings.

1.20 2.3.j Answer

By the book [5], there is often a strong inverse correlation between recording volume (e.g., amplitude) and localization error. When a sound is louder, acoustic sensors (e.g., the four fixed microphones in our case) capture a cleaner, stronger signal, leading to greater accuracy, while low-volume or distorted recordings should result in higher location error. The figure 7 below partly demonstrates that this statement holds true in practice, though it heavily depends upon how well the model was trained to handle sudden RMS variations. In our case, there is a visible positive correlation between acoustic volume and localization error that, at times, shifts to an inverse correlation. For example, at $t = 1.6s$, achieving a peak of $RMS = 0.22$ (which is a medium-height RMS peak for the blue bird), the localization error decreases to a good extent, although this is not sufficient to get a completely accurate prediction. Whereas at $t = 8.7s$, marked by a double-peak RMS body of $RMS = 0.15$, the localization error increases by a great extent, clearly marking the positive correlation.

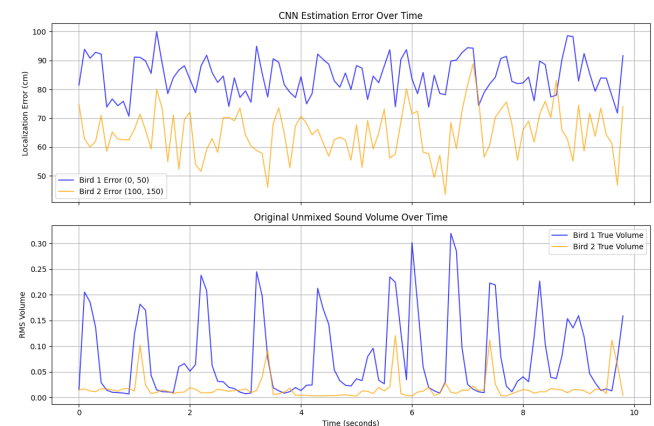


Figure 7: Location error correlation with the sound volume

REFERENCES

- [1] Andre Araujo, Wade Norris, and Jack Sim. “Computing Receptive Fields of Convolutional Neural Networks”. In: *Distill* 4.11 (2019), e21. DOI: 10.23915/distill.00021. URL: <https://distill.pub/2019/computing-receptive-fields>.
- [2] Bohrium Sciencepedia. *Effective Receptive Field*. https://www.bohrium.com/en/sciencepedia/feynman/keyword/effective_receptive_field. Accessed: 2026-05-27.
- [3] Srinath Bulusu et al. “Generalization capabilities of translationally equivariant neural networks”. In: *Physical Review D* 104 (7 Oct. 2021), p. 074504. DOI: 10.1103/PhysRevD.104.074504. URL: <https://aps.org>.
- [4] Taco S. Cohen and Max Welling. “Group Equivariant Convolutional Networks”. In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, 2016, pp. 2990–2999. URL: <https://arxiv.org>.
- [5] Gabriel Jekaterýńczuk and Zbigniew Piotrowski. *Sound Source Localization and Detection Methods*. <https://encyclopedia.pub/entry/53167>. Accessed: May 31, 2026. 2023.
- [6] Wenjie Luo et al. “Understanding the Effective Receptive Field in Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016.